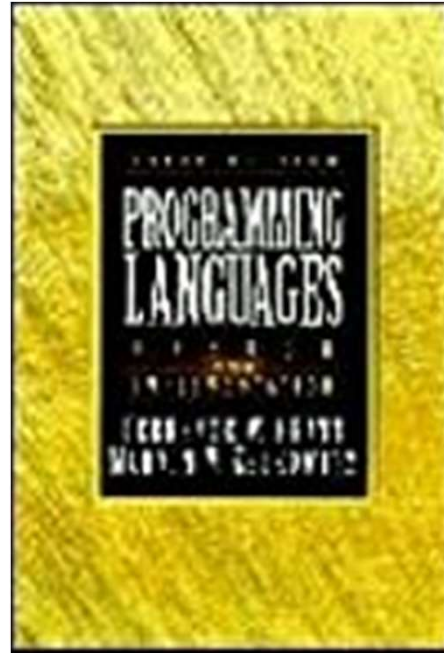




Controle de Subprograma

Interação entre Subprogramas

Parte 1 de 3

Controle de Subprograma (parte 1 de 3)

Conteúdo

2

- Controle de sequência de subprograma
 - ▣ Subprogramas simples e recursivos.
- Atributos de controle de dados
 - ▣ Nomes e ambientes de referências,
 - ▣ escopo estático e dinâmico
 - ▣ estrutura de blocos

Controle de Sequência de Subprograma

3

- Mecanismos simples para sequenciamento entre dois subprogramas
- Como um subprograma invoca outro e como o subprograma chamado retorna ao chamador.
 - ▣ Uso dos comandos **call** e **return**.

Call e return são comuns a muitas linguagens.

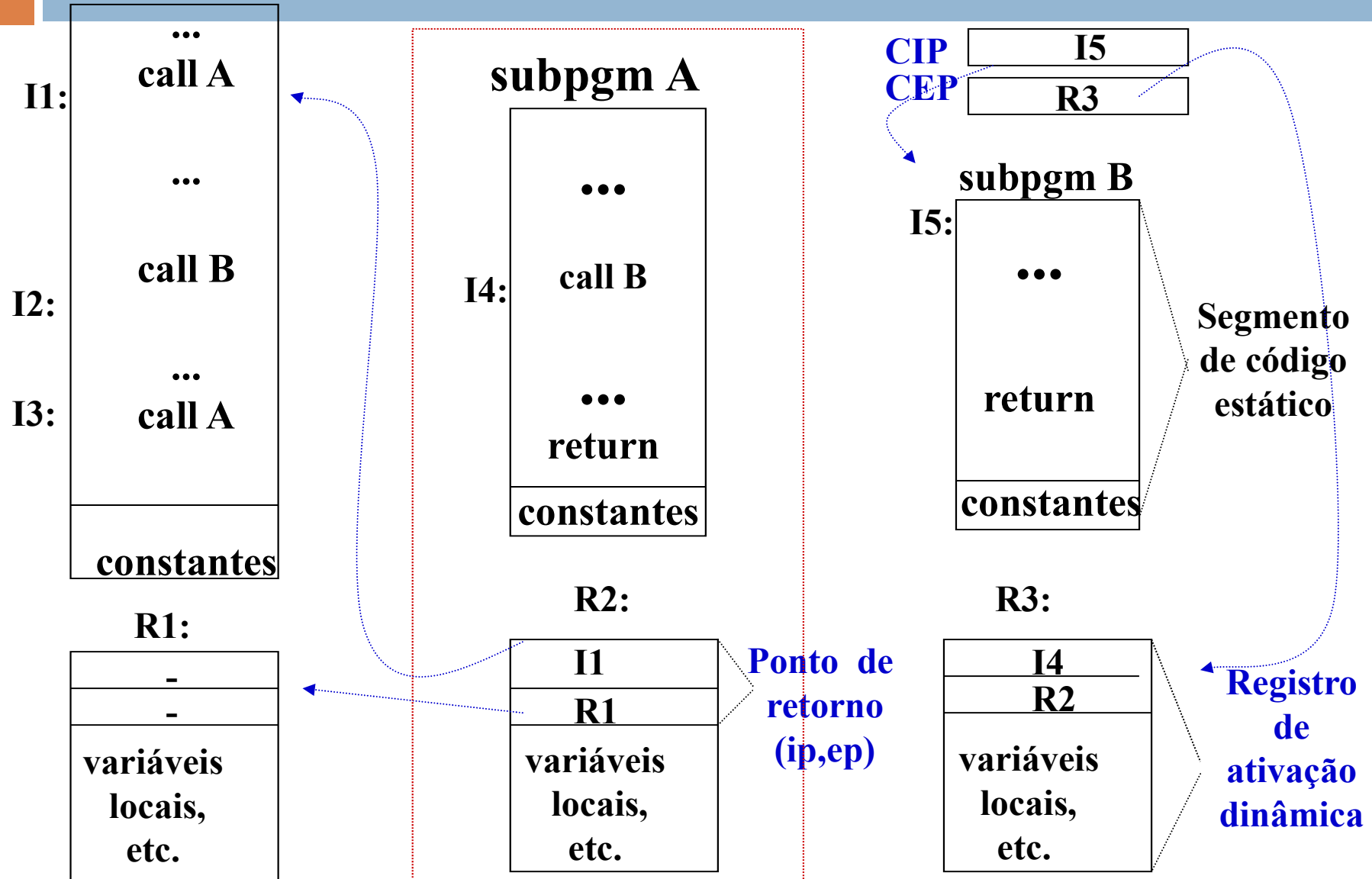
Estrutura de controle call-return

pgm principal

área de ativação

Instrução e ambiente correntes

4



Call/return em Subprogramas

Ponteiro da Instrução Corrente (CIP)

5

- O tradutor gera um código executável e o armazena no segmento de código, na área de ativação.
 - ▣ Esse código é um conjunto de instruções.
 - ▣ Essas instruções são executadas pelo hardware ou por interpretador simulado por software.
 - ▣ Em qualquer ponto durante a execução, alguma instrução está sendo executada ou próximo de sê-lo.
 - ▣ Essa instrução é chamada instrução corrente. A variável CIP é um apontador para ela. O interpretador faz o seguinte:
 - Pega a instrução apontada pelo CIP.
 - Atualiza o CIP para apontar para a próxima instrução.
 - Executa a instrução (que pode alterar o CIP).

Call/return em Subprogramas

Ponteiro do Ambiente Corrente - CEP

6

- Todas ativações do subprograma **usam o mesmo segmento de código** e registros de ativações **distintos**.
- Necessário saber qual registro de ativação está em uso.
 - ▣ CEP aponta para o registro de ativação em uso.
- O registro de ativação representa um **ambiente de referência local** do subprograma.
- Um segmento de código e um registro de ativação são criados para o programa principal:
 - ▣ O CIP aponta para a instrução corrente no segmento de código.
 - ▣ O CEP aponta para o registro de ativação.
- Para **executar** uma instrução call, cria-se um registro de ativação para a chamada, junto com um par (CIP,CEP).

Call/Return em Subprogramas

Ponteiro do ambiente corrente - CEP

7

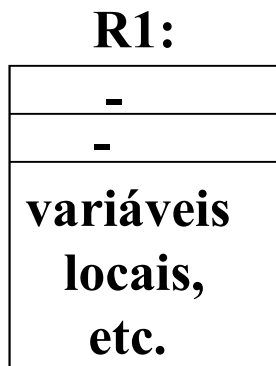
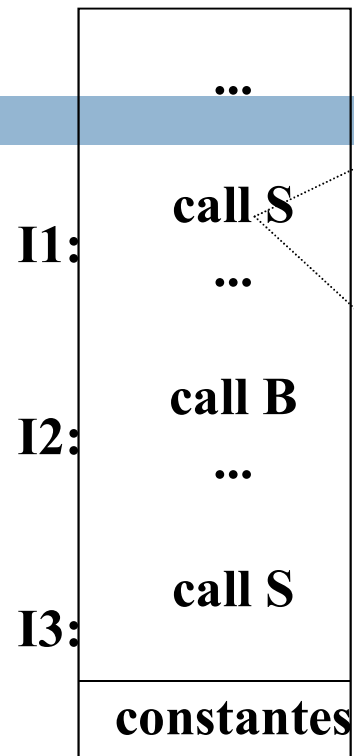
- O subprograma **chamador salva** os valores do (CIP,CEP) **antes de atualizar** com os **novos valores** do programa **chamado**.
 - ▣ Os valores de (CIP,CEP) são **salvos** no registro de ativação do subprograma **chamado**, no OD chamado **“ponto de retorno”**.
 - ▣ O “ponto de retorno” contém dois ponteiros **(ip,ep)**, que representam os **velhos valores** de CIP e CEP.
 - Há apenas um par (CIP,CEP) para cada programa em execução.
- Quando uma instrução *return* é alcançada, os **velhos valores** do par (CIP,CEP) **são restabelecidos**.

Subprogramas Recursivos

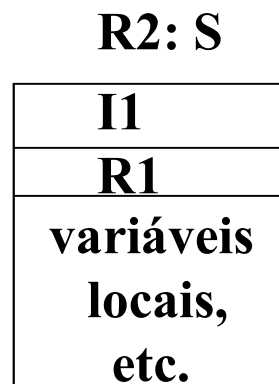
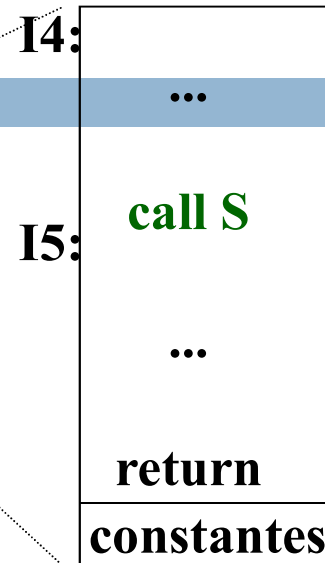
8

- Um subprograma recursivo chama a si próprio ou inicia uma cadeia na qual algum elemento chama o subprograma inicial.
- O modelo com CIP e CEP é geral e suficiente para **permitir recursão.**
- Seja S um subprograma recursivo. A cada chamada de S , um **novo registro de ativação para S é criado.**

programa principal



subprograma S

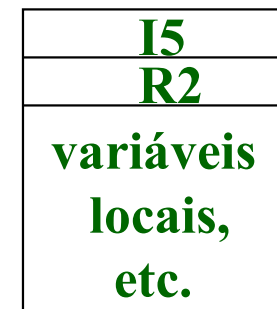


Ponto de retorno (ip,ep)

Segmento de código estático



R3: S



Registro de ativação dinâmica

Chamada recursiva de S: encadeamento dinâmico de registros de ativação

Atributos de Controle de dados

10

- Estão relacionados ao acesso aos dados de diferentes pontos durante a execução do programa.
 - **Dados precisam ser providos para a operação em realização, na sequência de execução.**
- As características de controle de dados da LP
 - Como os dados são providos para cada operação?
 - Nome e ambiente de referência
 - Como o resultado de uma operação pode ser salvo e recuperado depois, como operando de outra operação?
 - Dados e ambientes de referência

Nomes e Ambiente de Referência

11

- Modos de OD se tornar disponível para uma operação
 - ▣ Transmissão direta (usado em expressões):
 - Um OD computado em um operação pode ser transmitido diretamente para outra operação. Como $2*Z$ na expressão
$$X:=Y+2*Z;$$
 - É alocado um armazenamento temporária para o OD (anônimo)
 - ▣ Referenciando através de um OD nomeado.
 - Dá-se um nome para um OD, quando ele é criado, e esse nome é usado para designá-lo como operando de uma operação.
 - Um OD pode ser criado como componente de um outro OD, nesse caso o nome do OD maior e o nome do componente são usados na seleção.

O problema central é o significado dos nomes, em controle de dados.

Nomes e Ambiente de Referência

Elementos que Podem ser Nomeados

12

Categorias de nomes em LP:

1) variáveis

2) parâmetros formais

3) subprogramas

4) tipos definidos

5) constantes

6) rótulos

7) exceções

8) operações primitivas: +, sqrt, etc.

9) literais constantes: 3.5, 17, etc.

Referência nas categorias 4 a 9: nomes são resolvidos em tempo de tradução.

Casos especiais:

- ▣ exceções via encadeamento dinâmico em ADA.
- ▣ goto referenciando rótulos em outros subprogramas, como em Pascal.
- ▣ símbolos de operações primitivas, em SNOBOL4, podem ser redefinidos.

Apenas as categorias 1 a 3 serão objeto de estudo nesse tópico.

Nomes e Ambiente de Referência

Associações e Ambientes de Referência

13

- Associação: **ligação** de **identificador** a **OD** ou **subprograma**.
- Durante a execução do programa observa-se:
 1. variáveis declaradas
 - identificadores são associados aos OD.
 2. nomes de subprogramas chamados
 - são ligados às suas definições.

Identificador é um nome simples

Nomes e Ambiente de Referência

Associações e Ambientes de Referência

14

3. o programa invoca operações de referenciamento
 - Qual o OD ou subprograma associado ao identificador?
 - Para $A := B + FN(C)$; 4 operações de referenciamento são necessárias para recuperar os OD (A, B, C) e subprograma FN envolvidos.
4. a cada chamada de um subprograma, um novo conjunto de associações é criado para ele:
 - nome de parâmetros e nomes de variáveis declaradas são associadas com OD. Pode-se criar novas associações para os subprogramas declarados aninhados.

Nomes e Ambiente de Referência

Associações e Ambientes de Referência

15

5. Enquanto um subprograma executa, ele usa operações de referenciamento para determinar OD ou subprogramas associados aos identificadores.
 - Algumas referencias podem ser as criadas por associações na entrada do subprograma.
 - Outras podem ser as associações criadas no programa principal.
6. Quando o subprograma retorna o controle ao programa principal, suas associações são destruídas (ou tornam-se inativas).
7. Quando o controle retorna ao programa principal, a execução continua como antes, usando as associações criadas no início da execução.

Nomes e Ambiente de Referência

Principais Conceitos de Controle de Dados

16

- Associados ao padrão de **criação, uso e destruição** de associações:
 - ▣ Ambiente de referência
 - ▣ Visibilidade
 - ▣ Escopo dinâmico
 - ▣ Operação de referenciamento
 - ▣ Referências local, não local e global

Nomes e ambiente de referência

Ambiente de Referência

17

- Conjunto de **ligações** de identificadores **disponíveis** para **referenciamento**:
 - ▣ OD e subprogramas visíveis ao subprograma (ou programa), disponível para uso durante a sua execução.
 - ▣ O ambiente de referência de um subprograma é, em geral, invariante.
 - ▣ Os valores nos OD podem mudar, mas a ligação de nomes com os OD e subprogramas permanecem constantes.

Nomes e ambiente de referência

Ambiente de Referência

18

- Pode ter os seguintes componentes:
 - ▣ **ambiente local**
 - contexto **criado pelo registro de ativação** do subprograma.
 - ▣ **ambiente não-local**
 - associações usadas dentro do subprograma mas **não criadas durante a sua ativação**.
 - ▣ **ambiente global**
 - contido no ambiente de referência não-local mas criado pela ativação do **programa principal**.
 - ▣ **ambiente predefinido**
 - conjunto de associações de identificadores **providos pela LP**. Qualquer programa ou subprograma pode usa-los.

Nomes e Ambiente de Referência

Visibilidade

19

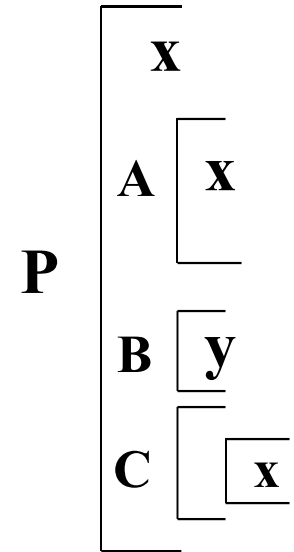
- Uma associação é **visível** dentro de um subprograma se faz **parte do seu ambiente de referência**.
- Uma associação **existente** está **escondida** se ela **não faz parte do ambiente de referência** do subprograma corrente.
- Se o subprograma **redefine** um identificador em uso em outra parte, a associação **atual fica visível** e as **anteriores são escondidas**.

Nomes e Ambiente de Referência

Escopo Dinâmico

20

- Cada **associação** tem um **escopo dinâmico**.
- O **escopo** da associação é a **parte da execução** do programa durante a qual **ela existe como parte de um ambiente de referência**.
- Portanto é o **conjunto de ativações** de subprograma no qual **ela é visível**.



Nomes e Ambiente de Referência

Operação de referenciamento

21

□ Assinatura:

op_ref: identificador x
amb._referência → OD ou
definição de subprograma.

- Dado um identificador e um ambiente de referência, encontra a associação adequada para o identificador no ambiente.
- **Retorna** o OD ou a definição do subprograma associado.

Referência:

- **local** – op_ref acha a associação no ambiente local.
- **não-local** – op_ref acha a associação no ambiente não-local.
- **global**: idem, no global.
- Vamos usar **global** e **não-local** de forma indistinta.

Nomes e Ambiente de Referência

Referência local, não local e global

22

```
program main;  
  var A,B,C: real;  
  procedure Sub1(A:real);  
    var D:real;  
    procedure Sub2 (C:real);  
      var D: real;  
      begin C:=C+B;  
    end;  
    begin Sub2(B);  
  end;  
begin  
  Sub1(A);  
end.
```

- **Ambiente de referência Sub2**
 - ▣ Local: C e D.
 - ▣ Não local: A e Sub2 em Sub1; B e Sub1 em main.
- **Ambiente de referência Sub1**
 - ▣ Local: A, D e Sub2.
 - ▣ Não local: B, C e Sub1 em main.
- **Ambiente de referência main**
 - ▣ Local: A, B, C e Sub1.
- **Ambiente predefinido**
 - ▣ write, read, sqrt, MAXINT, etc.

A, D e C são declarados duas vezes mas somente uma associação é visível por vez.

Nomes e Ambiente de Referência

Alias para OD

23

Nomes pelos quais um OD pode ser referenciado no mesmo ambiente de referência. Exemplos:

- ▣ **passagem de parâmetro por referência**: nome do parâmetro formal é visível no subprograma ativado embora o nome atual também o seja.
- ▣ **mais de um caminho**, via ponteiro, para o mesmo OD, no mesmo escopo.

- ▣ Dificulta entender o programa e para o implementador da LP otimizar o código:
 $x := a + b; y := c + d; /*x, c \text{ alias}*/$
- ▣ Enfoque atual: limitar alias.

```
var x: real;  
  sub (var y: real)  
    (* x e y referenciam o  
       mesmo objeto *)  
  sub(x) ;
```

Escopo Estático e Dinâmico

24

- Definições:
 - ▣ **Escopo estático**: parte do **texto do programa** onde o uso de um identificador é uma referência a uma particular declaração do identificador.
 - ▣ **Escopo dinâmico**: conjunto de **ativações de subprogramas** no qual uma associação é visível durante a execução.
- Regras de escopo estático **relacionam referências** com declarações de **nomes no texto** do programa.
- Regras de escopo dinâmico **relacionam referências** com associações a nomes, **durante o curso dinâmico de execução do programa**.
- Essas duas regras de escopo precisam ser consistentes.

Lisp, Snobol4 e APL são interpretados e não usam regras de escopo estático

Estrutura em Bloco

25

□ **Definição**

- Em uma LP estruturada em bloco, cada programa ou subprograma é organizado como um conjunto de blocos aninhados.
- A principal característica de um bloco é que ele introduz um novo ambiente de referência local.
- Um bloco é composto de declarações seguidas por comandos.

□ **Um bloco nomeado é um subprograma.**

procedure P(A:real); <declarações>; <comandos>;

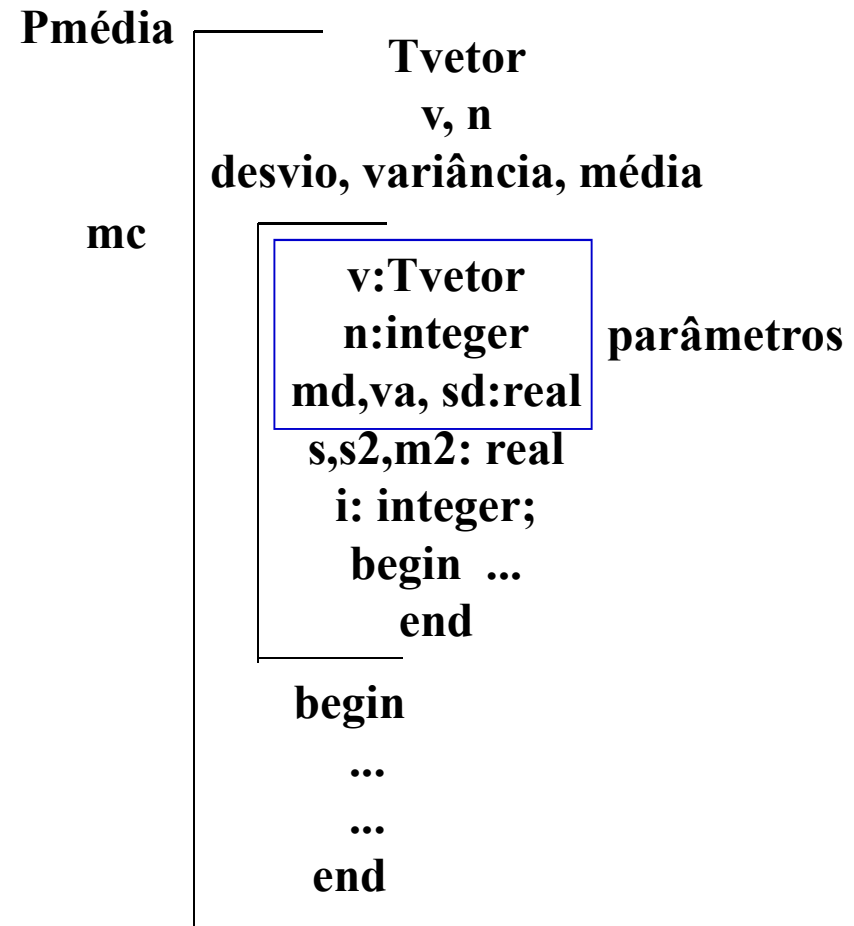
Esse conceito foi introduzido pelo ALGOL 60

Estrutura em Bloco

Exemplo de Bloco

26

```
program Pmedia(input, output)
type Tvetor=array (1..100) of real;
var v:Tvetor; n: integer;
    desvio, media, variancia: real;
procedure mc(var v:Tvetor; n:integer; var
md, va, sd: real);
var s, s2, m2: real; i:integer;
begin
    s := 0.0; s2 := 0.0 ;
    for i:=1 to n do
        begin
            s:= s + v[i];
            s2:=s2+v[i]*v[i];
        end ;
    md := s/n; m2 := md * md;
    va := s2/n - m2; sd := sqrt(va);
end ;
begin ... end;
```



Estrutura em Bloco

Declarações

27

- As declarações definem um ambiente de referência local para o bloco:
 - ▣ Parâmetros formais
 - ▣ Constantes
 - ▣ Variáveis
 - ▣ Subprogramas
- Esse ambiente é **invariante** durante a execução do subprograma.
- Declarações constituem a cabeça (**head**) do bloco enquanto comandos formam o seu **corpo**.

Estrutura em Bloco

Exemplos de LP com Blocos

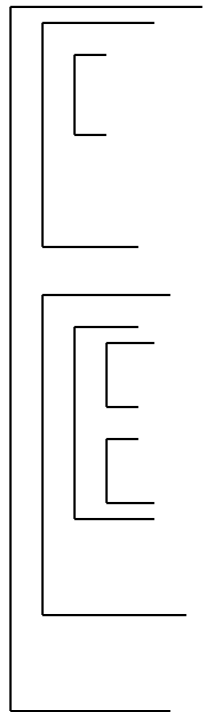
28

- ❑ Algol: blocos aninhados, com ou sem nome.
- ❑ Pascal: blocos aninhados nomeados.
- ❑ PL/I: blocos aninhados.
- ❑ ADA: blocos aninhados.
- ❑ C: cada subprograma é um único bloco que pode conter blocos não nomeados aninhados. Tem facilidades para tratar **nomes não-locais**.

Estrutura em Bloco

Aninhamento

29



- Um programa forma um bloco principal
 - ▣ Cada subprograma é um bloco.
 - ▣ Um subprograma pode declarar outros subprogramas.
 - ▣ Subprogramas são blocos com nomes.
 - ▣ Pode haver bloco sem nome (LISP, Algol, etc).
 - ▣ Há blocos nomeados que não são subprogramas (LISP).
- Blocos podem declarar qualquer objeto.

Estrutura em Bloco

Regras de Escopo Estática

30

□ Referência Local

Qualquer referência a um identificador, no corpo de um bloco, se o identificador for declarado no bloco, é uma referência local.

□ Referência não local

- Se o identificador não é declarado no bloco.
- A busca pelo identificador vai do presente bloco **interno até** o bloco mais **externo**.
- Não encontrando, busca entre os termos pré-definidos pela LP.
- Não encontrando, considera um erro.

Estrutura em Bloco

Regras de Escopo Estática

31

- **Declarações em sub-blocos aninhados**

Ficam escondidas, encapsuladas, dos blocos mais externos .

- **Nome de bloco (usualmente um subprograma)**

Faz parte do ambiente do bloco que o declara.

- **Parâmetros formais**

Fazem parte do ambiente de referência local do subprograma que os declara.